



Media Type Registry 配置(book + movie)



Media Type Registry 的 P0 配置。不变量 #2:业务代码禁止 `if media_type === "book"` ,所有差异化能力通过 Registry 暴露。

§1 类型定义

```
// lib/registry/types.ts

export type MediaTypeId = "book" | "movie" | "tv" | "music"
| "game" | "comic"

export type StatusGroup = "to_do" | "in_progress" | "complete" | "abandoned"

export type StatusOption = {
  key: string // 内部 key
  labelKey: string // i18n key
  isTerminal: boolean // 终态(用于完成统计)
  isActive: boolean // 进行中("继续阅读")
  group: StatusGroup
}

export type ProgressModelId =
  | "book_page_progress"
  | "watch_log"
  | "tv_linear_progress"
  | "audiobook_minute_progress"
  | "comic_chapter_progress"
  | "podcast_episode_progress"
```

```

    | "game_hour_progress"

export type ProviderId =
    | "openlibrary" | "googlebooks" | "tmdb" | "manual"
    | "douban" | "letterboxd" | "trakt"

export type SearchTextConfig = "chinese_jieba" | "english"
    | "simple"

export interface MediaTypeRegistry {
    id: MediaTypeId
    labelKey: string
    iconEmoji: string

    // 状态机
    statusOptions: StatusOption[]
    defaultStatus: string

    // 评分(P0 统一 5~50 步长 5)
    ratingScale: { minX10: 5; maxX10: 50; stepX10: 5 } | null

    // 进度模型
    defaultProgressModel: ProgressModelId
    supportedEditionTypes: string[]
    editionTypeToProgressModel: Record<string, ProgressModelId>

    // Provider 优先级
    searchProviders: ProviderId[]
    manualEntryAllowed: true

    // Annotation 位置约束
    annotationLocationKind: "book" | "video"

    // 搜索配置选择器
    pickSearchConfig: (text: string) => SearchTextConfig

    // 公开主页摘要字段

```

```
publicSummaryFields: string[]
}
```

§2 公用工具

```
// lib/registry/search-config.ts

export function pickSearchConfigByCJKRatio(text: string): SearchTextConfig {
  if (!text || text.trim().length === 0) return "simple"
  const cjkRegex = /[\u4e00-\u9fff\u3040-\u309f\u30a0-\u30ff\uac00-\ud7af]/g
  const cjkMatches = text.match(cjkRegex)?.length ?? 0
  const ratio = cjkMatches / text.length
  if (ratio >= 0.3) return "chinese_jieba"
  if (/[a-zA-Z]/.test(text)) return "english"
  return "simple"
}
```

§3 Book Registry

```
// lib/registry/book.ts

import type { MediaTypeRegistry } from "../types"
import { pickSearchConfigByCJKRatio } from "../search-config"

export const bookRegistry: MediaTypeRegistry = {
  id: "book",
  labelKey: "media.book.label",
  iconEmoji: "📖",

  statusOptions: [
    { key: "want_to_read", labelKey: "media.book.status.want_to_read", isTerminal: false, isActive: false, group: "to_do" },
    { key: "reading", labelKey: "media.book.status.reading" }
  ]
}
```

```

ding",      isTerminal: false, isActive: true,  group: "in_
progress" },
    { key: "finished",      labelKey: "media.book.status.fin
ished",      isTerminal: true,  isActive: false, group: "com
plete" },
    { key: "abandoned",      labelKey: "media.book.status.aba
ndoned",      isTerminal: true,  isActive: false, group: "aba
ndoned" },
  ],
  defaultStatus: "want_to_read",

  ratingScale: { minX10: 5, maxX10: 50, stepX10: 5 },

  defaultProgressModel: "book_page_progress",
  supportedEditionTypes: ["paperback", "hardcover", "eboo
k", "audiobook"],
  editionTypeToProgressModel: {
    paperback: "book_page_progress",
    hardcover: "book_page_progress",
    ebook:      "book_page_progress",
    audiobook:  "audiobook_minute_progress",
  },

  searchProviders: ["openlibrary", "googlebooks", "manua
l"],
  manualEntryAllowed: true,

  annotationLocationKind: "book",
  pickSearchConfig: pickSearchConfigByCJKRatio,

  publicSummaryFields: ["canonical_title", "first_release_y
ear", "cover_url"],
}

```

§4 Movie Registry

```
// lib/registry/movie.ts

import type { MediaTypeRegistry } from "../types"
import { pickSearchConfigByCJKRatio } from "../search-config"

export const movieRegistry: MediaTypeRegistry = {
  id: "movie",
  labelKey: "media.movie.label",
  iconEmoji: "🎬",

  statusOptions: [
    { key: "want_to_watch", labelKey: "media.movie.status.want_to_watch", isTerminal: false, isActive: false, group: "to_do" },
    { key: "watching", labelKey: "media.movie.status.watching", isTerminal: false, isActive: true, group: "in_progress" },
    { key: "watched", labelKey: "media.movie.status.watched", isTerminal: true, isActive: false, group: "complete" },
    { key: "abandoned", labelKey: "media.movie.status.abandoned", isTerminal: true, isActive: false, group: "abandoned" },
  ],
  defaultStatus: "want_to_watch",

  ratingScale: { minX10: 5, maxX10: 50, stepX10: 5 },

  defaultProgressModel: "watch_log",
  supportedEditionTypes: ["theatrical", "tv_cut", "director_cut"],
  editionTypeToProgressModel: {
    theatrical: "watch_log",
    tv_cut: "watch_log",
    director_cut: "watch_log",
  },
}
```

```

searchProviders: ["tmdb", "manual"],
manualEntryAllowed: true,

annotationLocationKind: "video",
pickSearchConfig: pickSearchConfigByCJKRatio,

publicSummaryFields: ["canonical_title", "first_release_year", "cover_url"],
}

```

§5 Registry 统一入口

```

// lib/registry/index.ts

import { bookRegistry } from "../book"
import { movieRegistry } from "../movie"
import type { MediaTypeId, MediaTypeRegistry } from "../types"

const registries: Record<MediaTypeId, MediaTypeRegistry | undefined> = {
  book: bookRegistry,
  movie: movieRegistry,
  tv: undefined, // P1
  music: undefined, // P2+
  game: undefined, // P2+
  comic: undefined, // P2+
}

export function getRegistry(id: MediaTypeId): MediaTypeRegistry {
  const r = registries[id]
  if (!r) {
    throw new Error(`Registry for media type "${id}" is not enabled in this phase.`)
  }
  return r
}

```

```

}

export function listEnabledMediaTypes(): MediaTypeId[] {
  return (Object.keys(registries) as MediaTypeId[]).filter
    ((id) => !!registries[id])
}

```

§6 单元测试要点

```

// __tests__/registry.test.ts

describe("Registry", () => {
  it("enables only book and movie in P0", () => {
    expect(listEnabledMediaTypes()).toEqual(["book", "movie"])
  })

  it("rejects non-P0 media types", () => {
    expect(() => getRegistry("tv")).toThrow()
  })

  it("status state machine is valid", () => {
    const r = getRegistry("book")
    expect(r.statusOptions.filter(s => s.isTerminal).length).toBeGreaterThan(0)
    expect(r.statusOptions.find(s => s.key === r.defaultStatus)).toBeDefined()
  })

  it("pickSearchConfig handles CJK / English / mixed", () => {
    expect(bookRegistry.pickSearchConfig("沙丘 第一部")).toBe("chinese_jieba")
    expect(bookRegistry.pickSearchConfig("Project Hail Mary")).toBe("english")
    expect(bookRegistry.pickSearchConfig("Dune 沙丘")).toBe("chinese_jieba")
  })
})

```

```
} )  
})
```

§7 扩展指引(P1+)

新增媒介(如 tv)的步骤:

1. 在 `MediaTypeId` 加字面量类型
2. 创建 `lib/registry/tv.ts`
3. 在 `registries` 注入
4. 添加 i18n bundle 项(见 i18n 子页)
5. 添加 progress_model seed
6. 添加 Provider Adapter
7. UI 添加流根据 Registry 自动暴露

禁止业务代码做 `if (mediaType === "tv")` 分流。差异化通过 Registry 字段暴露。